

Transaction Skill 生成能力介绍

1. 项目背景：为什么要建设 Transaction AI Studio

1.1 当前痛点：

1.1.1 Skill 生产门槛高、周期长

- 之前生成skill要通过：需求需经过“提需求 → 技术实现 → 调试与评测 → 发布”的长链路，反馈周期长。
- PM、运营即使更懂业务，也难以直接修改和验证 Skill，非技术角色也很难独立完成“写—测—发”的完整链路，导致平台能力增长速度受制于研发资源

1.1.2 Skill 资产分散、生成的质量难掌控

- 不同人维护同类Skill时，容易出现重复建设、版本漂移、能力冲突。
- 没有一个标准的准出门槛，不同的skill生成出来的质量参差，需要通过研发进行充分的 Review
- 每个人生成skill的经验差异化并且分散在各自的个人文档或临时版本中，缺少统一管理与复用入口。

1.1.3 Skill调试与发布困难

- 新建、修改、调试、回归验证、发布分散在多个工具或流程里。
- 调试的过程需要通过本地启动Skill Workbench，涉及到许多的Git操作，对非RD不友好
- Skill改动是否生效、是否稳定、是否可安全上线，缺少统一闭环。

1.2 前期方案设计思考：

为了解决这些问题，我们需要一个统一的平台进行skill的生成和调优，在前期产生了下面的思考：

1.2.1 选择怎样的Agent承担Skill开发任务

结论先行：我们最终选择Code Agent来作为Skill开发的任务。因为Skill开发本质上与代码开发高度相似：都需要在多份文档和代码中探索信息、理解已有能力、明确修改方案，再完成多文件编写、测试和迭代修复。而我们并没有自己搭建一个单独的Skill Agent，因为目前的 Sota Code Agent Harness已经足够强大，我们自己开发的周期长，并且也比不上其能力。而之

所以选择Codex，是因为其针对我们使用的GPT5系列的模型有单独的优化，是业界公认的顶级Code Agent之一，并且其开源合规，非常适合承担我们这次Skill开发的Code Agent任务。

1.2.2 如何在多 Pod 环境下保证Agent和文件系统稳定

Skill 开发依赖持续存在的工作区，但如果后端采用多Pod部署。用户的后续请求如果落到其他Pod，就无法访问原Pod的文件，导致任务无法继续。同时我们需要code agent来对文件系统进行操作，因此我们采用后端与工作区解耦的设计，将agent的工作区和文件系统跟实际的后端部署pod分离。

我们选择将我们的工作区放在TAE Sandbox中，将每个会话的 Codex、文件系统和命令执行放入独立环境，使后端只负责请求和任务编排，Agent 执行可以独立调度、隔离和替换。Sandbox有弹性周期，有可能会进行回收重启等，因此需要将相关的工作区文件保存到一个地方。引入ByteNas来对文件系统进行持久化的存储。

1.2.3 如何保证我们的skill生成的质量

为了保证Skill生成的质量，我们在Code Agent之上做了这些额外的拓展：

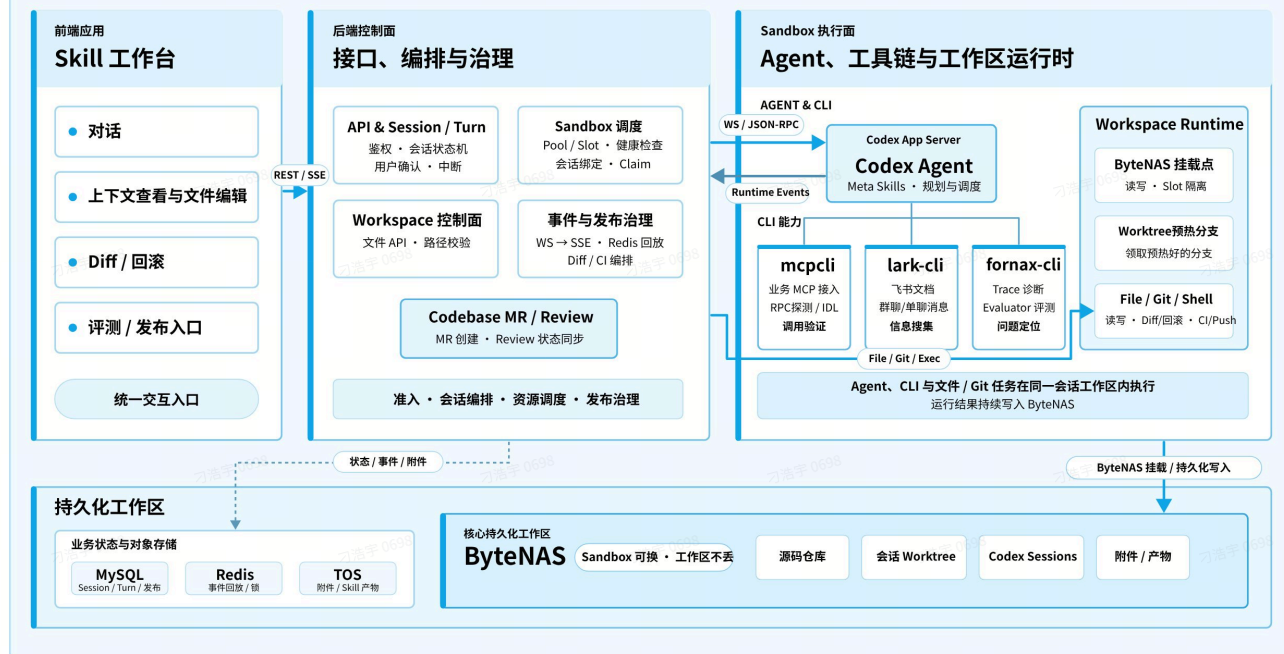


- **探索跟业务Skill相关资料：**Lark-CLI、MCP-CLI以及探索代码仓库的RPC接口的能力
- **通过真实的评测接口验证：**通过静态检查和线上评测接口，确认生成skill真实可运行
- **真实的Trace分析Skill执行：**通过Fornax-CLI获取执行Trace进行分析，让Agent自己找到不足
- **评估器对Skill结果进行验证：**使用Evaluator分别评估最终答案的正确性和 Skill 选择的正确性

2. 整体技术架构

2.1 整体架构

Transaction AI Studio 整体技术架构



2.2 各层职责与能力

2.2.1 前端

- **Agent对话交互**：支持用户通过自然语言描述需求，并持续与Agent沟通和确认。
- **工作区文件编辑**：支持浏览、查看和编辑当前Skill工作区中的文件。
- **AI变更审查展示**：展示Agent对文件产生的修改，并支持查看和撤销变更。
- **运行过程展示**：实时展示Agent的执行状态、工具调用和任务结果。
- **测试与评测入口**：支持发起Skill测试、查看运行过程和评测结果。
- **发布入口**：支持填写发布信息、发起发布并查看评审进度。

2.2.2 后端

- **Sandbox Session管理**：启动并检测Sandbox Session的健康状态，并在Sandbox失效时进行重建。
- **登录与鉴权**：完成用户登录、身份认证和接口访问权限校验。
- **Skill 读取**：提供Skill列表、详情及相关文件内容的查询能力。
- **Skill Session**：管理Skill开发会话及其消息、执行状态和用户交互。
- **工作区文件管理**：提供工作区文件的浏览、读取、编辑和删除能力。
- **AI 变更审查操作**：提供AI修改内容的查询和变更撤销能力。
- **测试与评测**：支持Skill测试、运行分析和效果评估。

- **发布管理**：完成发布前CI检查、提交评审以及发布状态跟踪。

2.2.3 Sandbox 层

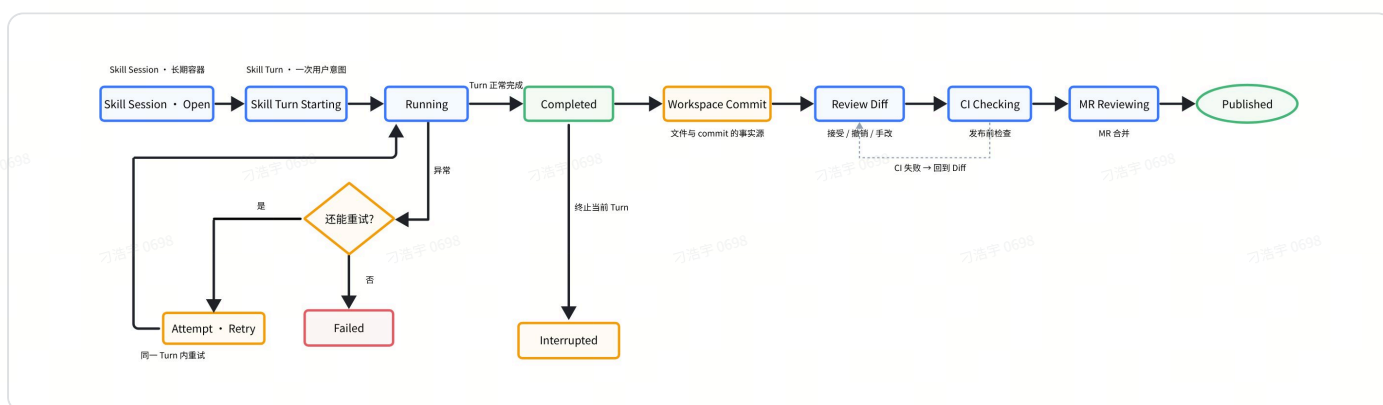
- **Agent隔离运行**：为每个Skill开发会话提供独立、安全的Agent执行环境。
- **Codex Server运行**：在Sandbox中运行Codex Server，接收后端指令并返回Agent执行结果。
- **Agent能力扩展**：为Agent提供飞书信息获取、代码检索、工具调用和运行分析等能力。
- **工作区工程操作**：支持Agent执行文件编辑、代码检索、命令运行和版本管理。
- **Skill调试与验证**：支持Agent在真实环境中测试Skill，并根据运行结果持续优化。

2.2.4 持久化层

- **MySQL业务状态存储**：保存Skill Session、任务执行和发布等业务状态。
- **Redis运行状态存储**：保存实时运行事件和短期协调状态，支持事件恢复和断线重连。
- **ByteNAS工作区存储**：持久化保存工作区文件，保证Sandbox重建后仍能恢复原有内容。
- **TOS Skill资产存储**：保存和提供全局Skill资产，供平台查询和使用。

3. 从一句需求到一个可发布 Skill：Meta Skill 驱动的Loop闭环

3.1 平台状态机



3.2 Skill Loop介绍：

3.2.1 Meta Skill的定位

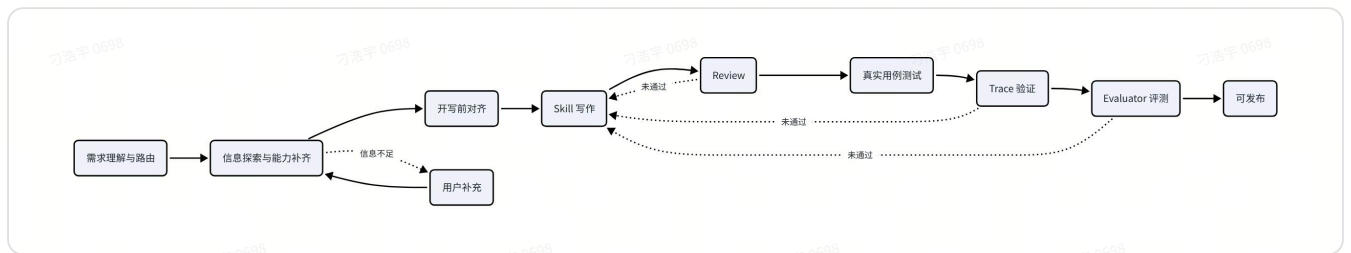
什么是Meta Skill？Meta Skill 是运行在Codex之上的流程编排能力，可以理解为用来“写Skill”的Skill。它将Skill创建、修改和调优过程中需要完成的意图理解、信息探索、方案对齐、Skill写作、Review、调试、评测和发布检查等工作，拆分为一组可以按需选择和组合的Skill，当

前做哪个阶段的任务，当前这个任务需要的产出和边界是什么，当前这个任务做完之后下一个节点应该去哪个任务。

在此基础上，我们通过Meta Skill管理完整的Skill Loop。Meta Skill会明确每个阶段任务需要的输入、应产生的结果以及进入下一阶段的条件；遇到信息不足或需要业务决策时暂停并请求用户确认，Review、调试或评测未通过时则回到相应环节继续修复。通过这种方式，Skill生成不再依赖Codex自由发挥，而是按照一套流程，从用户需求逐步推进到可发布状态。

3.2.2 完整Skill Loop

完整的 Skill Loop 从用户需求开始，依次经过需求理解与路由、信息探索与能力补齐、开写前对齐、Skill 写作、Review、真实用例测试、Trace 验证和 Evaluator 评测。只有各阶段均满足要求，Skill 才进入可发布状态。



- **需求理解与路由**：理解用户希望Skill解决的问题，并判断创建新Skill还是优化已有Skill。
- **信息探索与能力补齐**：判断是否有重复Skill，围绕目标收集业务资料和可用能力，补齐写Skill所需信息。
- **开写前对齐**：向用户确认目标、实现思路和验证方式，确认后再开始写作。
- **Skill 写作**：按照统一规范生成或修改Skill及其相关文件。
- **Review**：检查Skill的结构、触发边界、业务逻辑和文件一致性，并修复发现的问题。
- **真实用例测试**：使用用户提供或确认的真实Case，验证Skill能否在真实环境中运行。
- **Trace 验证**：检查目标Skill是否正确触发，以及实际执行过程是否符合预期。
- **Evaluator 评测**：评估最终答案是否正确，以及目标Skill是否被正确选择和使用。
- **可发布**：所有检查均通过且不存在阻塞问题后，Skill进入可发布状态。

Skill Loop 并不是单向流程：信息不足时会请求用户补充；Review、测试或评测未通过时，Meta Skill 会根据证据定位问题并回到相应环节继续修复，直到通过全部门禁或遇到需要用户决策的阻塞。

4. 困难与挑战

上述介绍了我们的整体技术架构以及我们生成skill的流程，接下来结合我们实际开发过程中的挑战来介绍一下我们遇到的困难和解决思路。

4.1 如何让Codex Server在Sandbox环境中稳定工作



- **启动耗时长**：Session创建涉及Sandbox、Codex和NAS工作区初始化，链路耗时较长。
- **运行实例不稳定**：Sandbox可能随时失效，Session不能依赖具体实例持续运行。
- **整体状态恢复**：文件、会话和事件状态分散在多个存储中，恢复时必须保持一致。
- **执行过程可能失败**：模型或Runtime异常可能导致Turn中断、重复或卡住。
- **连接随时可能中断**：Codex WebSocket、前端SSE等待链路都可能断开。

如何用有生命周期、可能失败的Sandbox，承载需要进行长链路Skill Session任务的Code Agent。

4.1.1 冷启动耗时：通过双层优化缩短启动链路

首先介绍Sandbox中的工作区结构。系统维护一份与远端Master持续同步的**基础仓库**；用户新建Session时，会以最新**Master Commit**为代码基线，创建独立的**Worktree**和**Session分支**，从而为每个会话提供相互隔离的开发空间。而为了持久化我们将整个工作区挂载到了**ByteNAS**上。受限于ByteNAS写入速度，每次全流程的创建会消耗特别多的时间，就会出现用户点击一次新建会话需要加载很久才能进入真正的模型问答中。为了解决耗时问题，我们做了如下优化

- **优化一**：使用**Shallow Clone**初始化基础仓库，仅保留**Master的浅层历史**；后续增量同步Master的最新变更，减少Git历史数据在ByteNAS上的写入，加快工作区创建。
- **优化二**：后台以最新**Master Commit**为基线，预创建并持续补充**Worktree Pool**。创建Session时，只需领取**预热Worktree**并创建独立分支，将Git同步和Worktree创建移出请求链路；**初始化失败**的Worktree会自动归还池中。

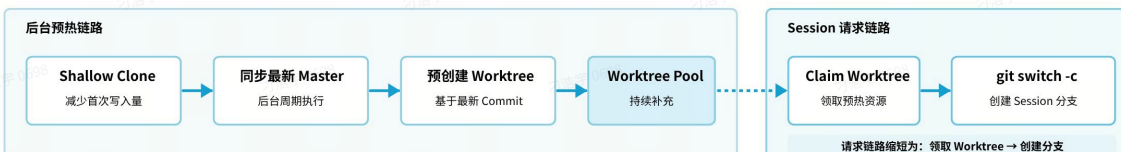
工作区准备链路：从同步创建到后台预热

将耗时的仓库同步与 Worktree 创建移出 Session 请求链路

改造前
Git 操作串行进入 Session 请求链路



改造后
后台持续预热，Session 只领取可用工作区





Session创建链路由“同步Master、创建 Worktree和分支”简化为“领取Worktree并创建分支”，显著缩短了Session创建耗时。

4.1.2 实例可能失效：通过稳定的Slot和镜像重建Sandbox和Codex

解决了Sandbox创建耗时长的问题，我们还发现Sandbox实例可能因过期、发布或异常失效，因此我们的Skill Session不能直接依赖某个Sandbox实例。

■ 解决思路：

为了将Skill Session跟Sandbox实例解耦，我们引入稳定的Slot，它对应ByteNAS中的/slots/<slot-id>子目录，将用户的会话跟ByteNAS存储结构绑定。新Session优先复用用户最近使用的Slot，同时通过哈希算法保证用户第一次使用时从ByteNAS中选择属于自己的Slot目录。

随后每一次在Sandbox实例启动时会挂载到这个Slot子目录下，这样用户无论Sandbox的实例怎么变化，最后操作的永远都是这份持久化存储的工作区。举个例子：用户第一次使用平台创建Skill Session时，我们通过哈希算法分配到用户到Slot-1，用户所有需要持久化的数据就都会存储到ByteNAS中/Slots/Slot-1目录下。

■ 启动Sandbox：

上文说到我们如何保证我们的工作区能够稳定重建，下面介绍我们的Sandbox启动的详细设计。

有状态部分：首先Slot通过ByteNAS持久化用户会话的有状态数据，包括基础仓库、各Session的Worktree和Codex会话记录。Sandbox实例更换后，新实例重新挂载原Slot，即可恢复代码、分支、工作区及历史对话。这部分我要详细介绍一下Codex会话恢复的机制：

Codex将完整会话记录追加写入Rollout JSONL，并通过SQLite建立查询索引。恢复时，只需根据Thread ID 回放对应的Rollout，即可重建对话上下文；SQLite索引可以重新生成，因此无需持久化。因此我们最后存储的只是Codex的Rollout JSONL文件。

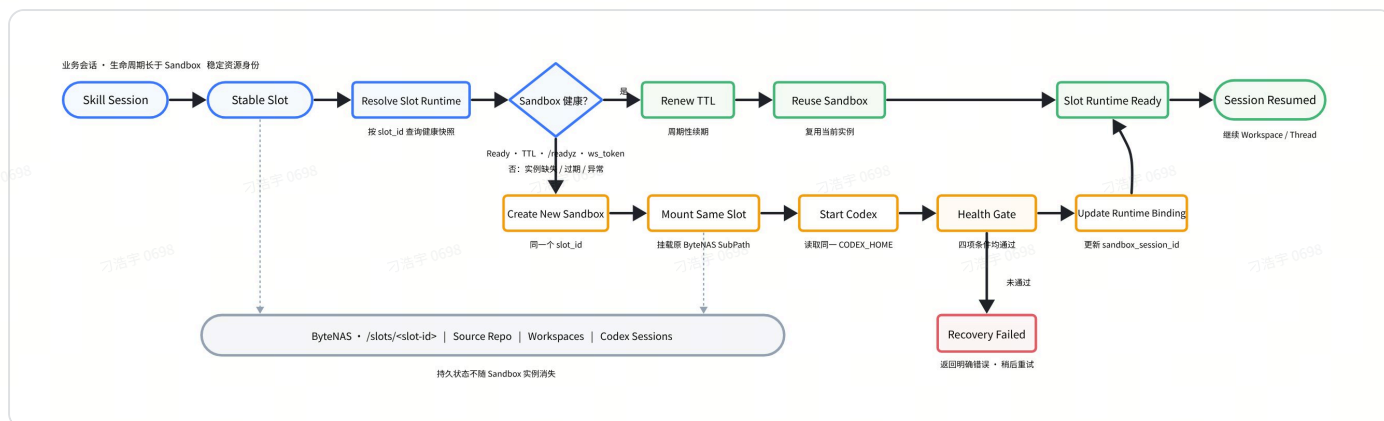
无状态部分：无状态的运行能力由版本化的ICM镜像提供。镜像固定Codex App Server、Lark CLI、MCP CLI、Fornax CLI及其依赖，并内置Meta Skills、全局指令和启动脚本，确保新Sandbox具备一致的运行环境。镜像负责恢复运行能力，Slot 负责恢复用户数据。

新Sandbox实例挂载原Slot后，系统会关联持久化的Codex会话目录，生成模型配置、WebSocket Token、Fornax Hook和AGENTS.md，同时完成Lark CLI鉴权配置与MCP Tool清单同步。随后启动 Codex App Server；健康检查通过后，后端即可重新建立WebSocket连接，并基于已有Codex Rollout继续会话。

■ Sandbox实例检测：

后台Sandbox Pool会周期性检查和调度实例，只有所需组件Ready、TTL充足并且Codex正常的实例才可使用。健康实例会定期续期；发现Slot对应的Sandbox实例缺失或不可用时，系统会在

创建新Sandbox、挂载原Slot下的ByteNAS目录并启动Codex，完成Sandbox实例的恢复和工作区的重新使用，保证在Sandbox实例失败后还能够有效的恢复之前的记录。



图：稳定 Slot 下的 Sandbox 重建与状态恢复



将Skill Session绑定到稳定的Slot上，避免Sandbox实例影响历史的会话

4.1.3 运行时异常与连接恢复



Codex runtime可能失败，模型请求可能临时失败，流式连接也可能中断，导致任务执行失败或前端丢失部分输出。

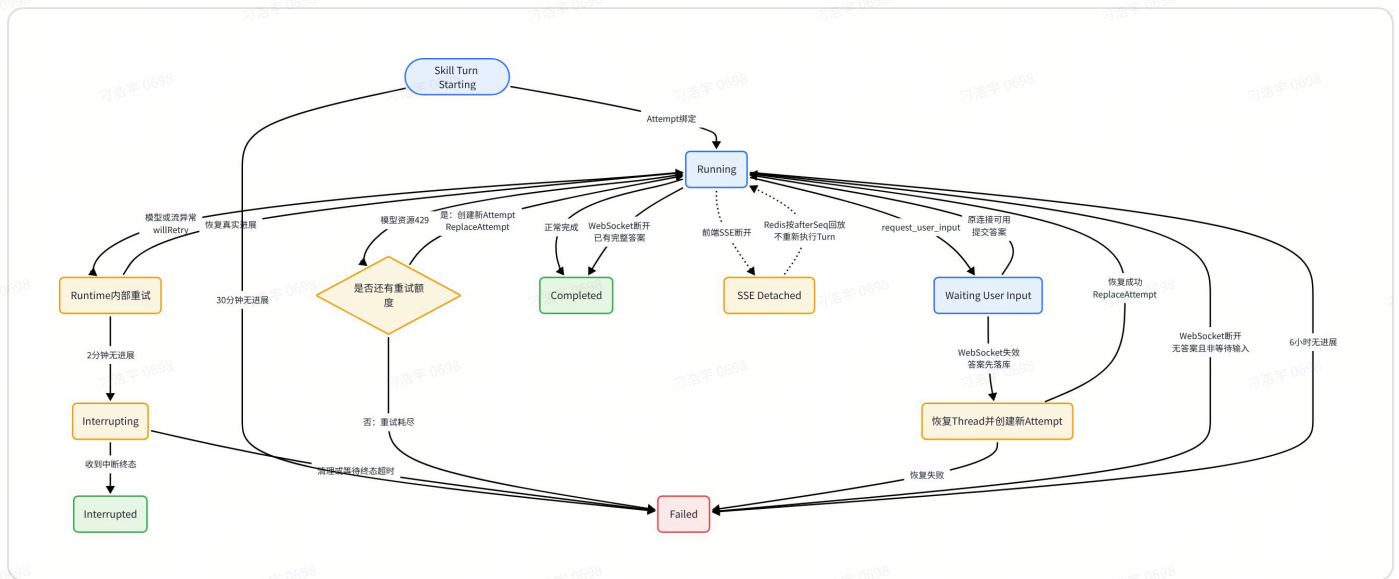
一次用户提交对应一个Skill Turn，一个Skill Turn可以包含多次Codex Attempt。系统按照“Runtime 自动重试 → Attempt 重试 → Redis 事件回放”分层处理异常：

- Runtime 自动重试：**Codex在当前 Attempt 内重试瞬时的模型请求和流式响应异常，不改变业务 Turn。
- Attempt 重试：**Runtime无法继续执行时，在同一个Skill Turn内创建新的Attempt，继续原任务，并通过状态校验保证只有最新Attempt可以更新结果。
- Redis 事件回放：**Codex事件按序写入 Redis，前端重连后从上次接收位置继续回放，只恢复页面展示，不重新执行任务。

三层机制解决的问题不同：**Runtime自动重试处理瞬时故障，Attempt 重试负责重新执行，Redis 回放负责恢复前端连接。**

异常场景	处理方式
模型请求失败或流式响应中断	Codex Runtime 在当前 Attempt 内自动重试，恢复后继续执行。
模型资源限流	Runtime 重试耗尽后，在原 Skill Turn 内创建新 Attempt；重试耗尽则结束任务。
Runtime 自动重试卡住	检测到长时间没有真实进展后主动中断，并将任务收敛到明确终态。
Codex WebSocket 断开	已产生完整回答则正常完成；正在等待用户输入则保留状态，并在用户提交后通过 Attempt 恢复；其他情况标记失败。
前端 SSE 断开	前端携带最后收到的事件序号重新连接，从 Redis 回放缺失事件，再继续接收实时
Turn 长期没有进展	后台恢复任务识别异常 Turn，并将其收敛为失败或已中断状态。

完整异常修复状态机



图：Skill Turn运行时异常处理与状态恢复

4.2 如何提升Codex寻找生成Skill所需资料的能力

完成了系统基础设施的建设后，需要聚焦如何更好的生成业务Skill。我们的Skill生成平台是为了服务于我们的业务Skill的生成，但是通用的Code Agent不存在这方面的知识。如何让Codex拥有创建业务Skill所需要的能力，是我们的第二个挑战。



- 业务信息分散：资料分布在多个平台
- 探索路径出错：Codex不知道如何探索
- 能力缺口无法自行补齐：会出现其无法决策或者缺少信息的情况

4.2.1 信息源分散：构建面向真实信息的多源探索能力

在我们生成一个业务Skill时，我们通常需要明确具体的业务流程是什么，是否有MCP Tool或者Rpc接口帮我们拿到具体的结果，最后的结果又该如何解释。这些信息分别散落在各个平

台中：例如**飞书文档**，**飞书群聊信息**或者**代码仓库**。因此我们为Codex提供了以下的拓展能力，让其跟我们所需要探索的空间打通。

我们在提供能力的同时，为我们的能力搭配了相关的操作skill，让Codex明确如何使用这些能力去探索到其需要的信息。详细介绍如下：

1. Lark-CLI:

我们通过将飞书官方提供的Lark-CLI放入我们的Sandbox中，来帮助我们完成飞书文档、群聊等内容的探索。

作用： 读取飞书文档、群聊和附件，帮助Codex获取业务流程、字段含义、讨论结论和实际案例。

如何使用： 由于Codex运行在共享Sandbox中，无法依赖机器上的默认飞书登录状态，相关Skill为文档和群聊设计了不同的身份入口：文档使用当前用户的授权读取，群聊使用已经加入目标群的机器人读取。Skill会先让Codex判断资料类型。对于飞书文档，先读取目录，再按相关章节逐步下钻；对于群聊，根据群聊ID、关键词和时间范围搜索相关消息；遇到图片或附件时，再进一步读取其中的内容。完成探索后，Codex会整理信息来源、业务规则、接口线索和待确认问题，供后续Skill编写使用。

2. MCP-CLI:

MCP-CLI是我们自建的Python CLI进程，其打通了MCP Server能够实现与MCP Sever的所需交互。

作用： 探索当前可用的MCP Tool，读取Tool的Schema、配置和RPC IDL，验证具体调用；当现有能力不足时，还可以根据RPC PSM和method查询BAM元数据、生成Tool Payload，并完成MCP Tool的创建、更新和发布。

如何使用： 相关Skill会先让Codex查询当前已授权的MCP Server和Tool Inventory，判断是否已经存在满足需求的Tool。找到候选后，会进一步检查Tool的输入参数、返回结构，以及底层RPC PSM和method是否与目标接口一致，必要时通过最小调用验证其真实能力。

如果代码仓库中已经找到了可用的RPC接口，但当前Inventory中没有对应Tool，Codex会使用MCP-cli根据RPC PSM和method从BAM获取接口元数据，自动生成MCP Tool Payload，而不是由模型手写IDL和参数结构。Payload校验和dry-run通过后，再向用户确认是否创建或更新Tool；得到确认后才执行发布并回读配置进行验证。

3. 代码仓库探索能力:

我们给定了一个代码仓库探索的skill，让Codex明确一个代码仓库按照什么路径去探索

作用： 从真实代码中确认业务逻辑、字段来源、判断条件和可用的RPC接口。

如何使用： Codex会先建立仓库结构，再从最终答案所需的数据出发，沿着IDL、RPC实现和下游调用链逐项查找字段来源、请求参数、返回结果和业务判断，并为结论保留代码证据。这样可以避免只搜索少量关键词就得出结论，也能将找到的RPC接口继续交给MCP-cli判断是否存在可直接使用的MCP Tool。



- **Lark-CLI:**为Codex提供飞书文档、群聊读取能力，帮助其获取业务流程、规则说明和讨论结论
- **MCP-CLI:**为Codex提供MCP Tool的发现、验证和补齐能力
- **代码仓库探索:**为Codex提供从真实代码中定位业务逻辑、字段来源和RPC接口的能力

4.2.2 探索容易偏航：通过需求清单和证据优先级约束探索路径

这一节我们继续聚焦我们最提出的问题，上一小节我们给予了我们的Codex充分探索的能力以及探索的指导。但是现在还存在一个问题：它不知道该怎么进行探索，胡乱的探索可能会造成Token和时间的双重浪费。为了解决这个问题我们做了如下的设计：

首先，我们引导Codex从用户的最终目标出发，拆解出生成Skill所必需的业务规则、数据字段、判断逻辑和执行能力，形成能力缺口清单。后续的每一次探索都必须对应清单中的某一项缺口，避免围绕无关信息不断扩展。

其次，我们关心如何去探索具体的能力缺口：我们为缺口探索指定了优先级，首先去探索用户提供的飞书信息、代码仓库以及通过MCP-CLI探索已有Tool，都查不到才能够去根据目前已有的Skill进行探索。

提出这个优先级的原因是在开发的过程中发现：如果不指定优先级的话模型会倾向于“抄”已有的Skill，它会先去检查目前已有的Skill中是不是有相近的业务逻辑和MCP Tool，然后直接使用。这些逻辑或者MCP Tool不一定会适配我们新生成的Skill导致其付出额外的探索时间，提高Skill生成的错误率。

最后，我们为Codex补充了交易的相关知识，例如不同的Domain下的Skill应该去探索哪些代码仓库。让其能在探索能力缺口的过程中，少走弯路的同时尽可能的符合我们的要求。

4.2.3 信息存在缺口：通过 Ask Human 和写作门禁完成探索闭环

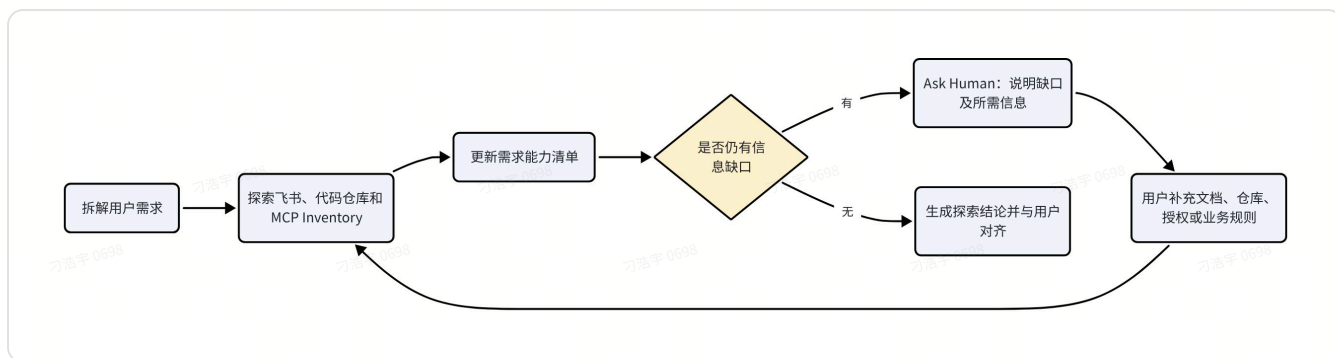
完成一轮探索后，Meta Skill会根据用户需求清单逐项检查业务规则、代码实现和MCP能力是否已经得到验证。此时，流程不会直接进入Skill 写作，而是根据检查结果进入两个分支。

如果仍有需求无法得到答案，Meta Skill会将其标记为信息缺口并触发Ask Human。系统需要向用户说明已经探索了哪些信息源、当前能够确认什么、仍然缺少什么，以及用户可以通过何种方式补充信息。

用户补充业务说明、文档、仓库信息或授权后，Meta Skill会保留已有探索结果，仅围绕尚未解决的缺口继续探索，并重新执行需求覆盖检查。这样可以形成“探索—发现缺口—请求补充—继续探索”的循环，避免在信息不足时通过模型推测补齐。

如果需求清单中的每项能力都已经得到验证，Meta Skill会先向用户输出需求理解、能力映射、依赖工具和实现边界。只有用户确认该结果符合预期后，流程才会进入Skill写作阶段。如果用户提出修改，则将新增或变化的内容重新纳入探索流程。

因此，进入写作阶段需要同时满足两个条件：一是用户需求中的每项能力都有真实依据和可行的实现方式；二是 Codex 输出的需求与能力映射已经得到用户确认。



图：信息缺口补充与用户对齐闭环

4.3 如何保证生成的Skill真实可运行并达到发布标准

在Codex进行了充分的探索之后，如何生成一个**符合标准的Skill**，并且怎么样确保我们生成的**Skill是可运行的、正确的**并且达到我们的**发布标准**，是我们第三个挑战。



- AI 如何生成一个标准Skill?
- 如何证明它在线上真实可运行?
- 如何证明它正确并且达到准出要求?

4.3.1 Skill Template：教 AI 写出标准 Skill

首先，我们需要解决“**AI应该如何编写Skill**”的问题。如果只向AI描述业务需求，不提供统一约束，不同任务生成的 Skill 很容易在目录结构、内容组织、工具调用和异常处理上产生较大差异，也难以被后续流程统一验证和发布。

为此，我们通过**Skill Template**向Codex明确一个标准Skill应该包含哪些内容，以及不同复杂度的Skill应该如何组织。**Template**不仅规定了**SKILL.md**的基础结构，还约束了触发描述、输入要求、执行流程、工具声明、异常处理、脚本、References、Testcase 和输出格式等内容。在实际写作过程中，Codex会根据业务复杂度选择对应的 Skill 类型，并按照Template生成 Skill 文件。

通过这一层，我们能够保证生成的Skill在结构、内容和工程规范上达到统一标准，为后续线上运行提供基础。

4.3.2 从真实运行到最终评测：Test、Trace与Evaluator闭环

为了确保写出的skill不只是符合格式规范，而是真实可用，我们在Codex生成Skill之后有一套完整的自动化评测流程。

i. 通过 Test 接口验证真实运行

Skill 生成完成后，Codex会先在独立Review时运行三项静态检查，对可以确定性判断的工程规范进行验证：

- **作者信息检查**：检查 SKILL.md中的author是否存在且格式正确。
- **描述相似度检查**：比较本次修改与已有Skill的描述，相似度达到90%时阻断，避免重复建设和触发范围冲突。
- **租户与输出格式检查**：检查supported_tenants和output format配置是否合法且不存在范围冲突。

静态检查通过后，Codex会使用用户提供或确认的真实 Test Case，通过统一测试脚本调用线上 Skill Test接口。请求会携带当前代码分支、目标Skill、租户和用户问题，使待测Skill在真实的模型、路由和工具环境中执行。

Test接口返回执行状态、最终答案和log_id，用于确认Skill能够被加载、触发并完成一次真实运行。该阶段只能证明 Skill “可以运行”，不能证明执行过程和最终结果正确，因此还需要基于 log_id进入后续Trace验证。

ii. 通过 Trace 定位问题并自动迭代

每次线上运行结束后，Codex都会根据本次运行的log_id使用Fornax CLI拉取完整Trace，检查目标Skill是否真正触发、是否执行了预期脚本和MCP Tool、关键参数是否来自真实输入，以及工具返回、业务判断和最终答案是否一致。

如果Trace发现问题，Codex会根据证据定位根因：使用与问题相关的能力重新查询资料、重新探索代码仓库等修改对应的Skill实现。

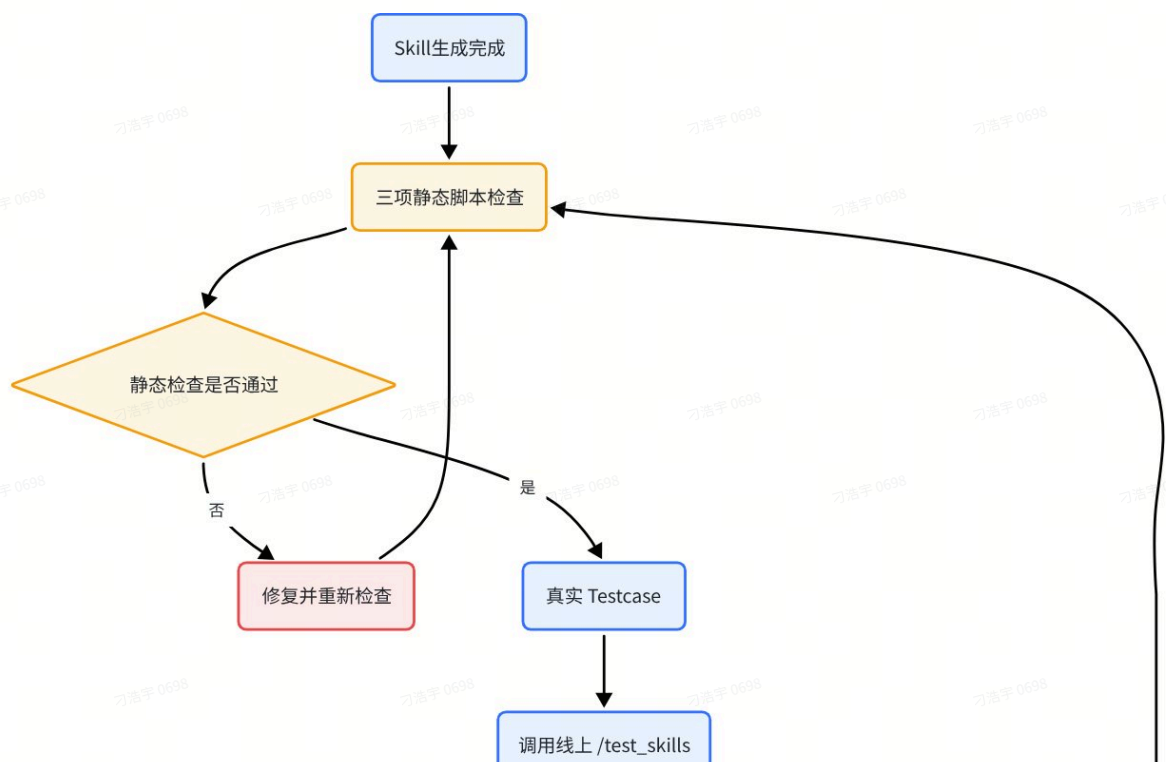
对于已有证据能够确定的问题，Codex会只修改最小必要范围，并使用同一个真实Case重新调用评测接口，再根据新的log_id验证Trace，形成“运行—定位—修复—复测”的自动迭代闭环。只有缺少业务事实、授权或存在多种无法判断的修复方案时，才会暂停并请求用户补充信息。

iii. 通过 Evaluator 完成最终验证

当线上评测结果和Trace执行过程均验证通过后，Codex会将真实问题、预期答案、线上最终答案以及Trace中记录的实际Skill使用情况提交给Fornax Evaluator下面两个评估器：

评估器Answer_correctness用于判断最终答案是否符合业务预期，评估器Skill_selection_correctionness用于判断目标Skill是否被真正选择和使用。这样可以同时避免“过程正确但答案错误”和“答案偶然正确但Skill选择错误”两类问题。

任一评估器失败，Codex都会继续根据Evaluator和Trace证据定位问题，并重新进入修复和复测流程。只有Test、Trace和两个Evaluator全部通过，Skill才会进入最终发布阶段。





图：从静态检查、线上运行、Trace自修复到Evaluator评测的完整闭环

- ✓ 通过Test接口证明Skill能够真实运行，通过Trace证明Skill按照正确路径执行，再通过双Evaluator证明执行结果和Skill选择均符合预期，最终保证Skill以正确的方法得到正确的结果。

5. 后续展望

5.1 接入知识库

前文提到，我们的Code Agent缺少交易领域的知识，因此在进行业务Skill的编写以及修改或者探索相关的MCP Tool或者代码仓库时会有很大一部分决策是根据模型自己的预训练知识。但是，其可能会带来很大的偏差。因此我们需要一个知识库让Codex在进行探索的过程中能够补充交易领域的知识来帮助其进行更好的探索，提升生成Skill的速度和质量。

5.2 定期删除磁盘文件

我们目前的工作区和Codex聊天记录通过存储到ByteNAS达到持久化的效果，但是ByteNAS有配额的上限，因此需要一个过期清理的机制来删除用户已经不使用的Skill Session相关的工作区和Codex聊天记录。

(注：部分内容可能由 AI 生成)